Sesión 8
MODELOS SEQ2SEQ Y ATENCIÓN

De secuencia a secuencia con mecanismos de alineación

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **1**

1. Modelos Seq2Seq y Atención **2**
1.1. Logro de la sesión . . . 2
1.2. Introducción: problemas de secuencia a secuencia . . . 2
1.3. Arquitectura Encoder-Decoder . . . 3
1.3.1. Visión general . . . 3
1.3.2. Encoder . . . 3
1.3.3. Decoder . . . 3
1.3.4. Limitación del contexto fijo . . . 4
1.4. Mecanismo de Atención . . . 4
1.4.1. Intuición . . . 4
1.4.2. Atención de Bahdanau (Additive Attention) . . . 4
1.4.3. Atención de Luong (Multiplicative Attention) . . . 5
1.4.4. Atención global vs. local . . . 5
1.5. Implementación paso a paso en PyTorch . . . 5
1.5.1. Preparación de datos . . . 5
1.5.2. Encoder . . . 6
1.5.3. Decoder con Atención (Luong) . . . 6
1.5.4. Modelo Seq2Seq completo . . . 7
1.5.5. Entrenamiento . . . 8
1.6. Métrica de Evaluación: BLEU . . . 9
1.6.1. Definición . . . 9
1.6.2. Cálculo . . . 9
1.6.3. Ejemplo numérico . . . 10
1.6.4. Interpretación . . . 10
1.7. Aplicaciones en la industria . . . 10
1.7.1. Traducción automática . . . 10
1.7.2. Summarization abstractivo . . . 10
1.7.3. Diálogo y chatbots . . . 10
1.7.4. Generación de preguntas (QA) . . . 11
1.8. Preparación para entrevistas . . . 11
1.8.1. Preguntas típicas . . . 11
1.8.2. Preguntas avanzadas . . . 11
1.9. Conexión con el resto del curso . . . 12
1.10. Resumen . . . 12

Capítulo 1

Modelos Seq2Seq y Atención

Las redes recurrentes vistas en la sesión anterior procesan secuencias, pero no están diseñadas para transformar una secuencia en otra de diferente longitud. Los modelos **Seq2Seq (Sequence-to-Sequence)** resuelven este problema mediante una arquitectura encoder-decoder, donde un codificador comprime la secuencia de entrada en un vector de contexto y un decodificador genera la secuencia de salida a partir de ese vector [citation:1][citation:5]. Sin embargo, el vector de contexto fijo se convierte en un cuello de botella para secuencias largas. El **mecanismo de atención** supera esta limitación permitiendo al decodificador acceder a todos los estados del encoder en cada paso [citation:4][citation:8]. Esta sesión cubre en profundidad ambas arquitecturas, sus fundamentos matemáticos, implementación y aplicaciones.

1.1. Logro de la sesión

Construir modelos de secuencia a secuencia (Seq2Seq) para tareas como traducción automática y summarization, comprendiendo el mecanismo de atención y las métricas de evaluación como BLEU.

1.2. Introducción: problemas de secuencia a secuencia

Muchas tareas de NLP requieren transformar una secuencia de entrada en una secuencia de salida de longitud potencialmente diferente. Ejemplos:

- **Traducción automática**: inglés → francés (longitud variable).
- **Summarization**: artículo largo → resumen corto.
- **Diálogo**: pregunta del usuario → respuesta del sistema.
- **Generación de preguntas**: texto → pregunta relevante.

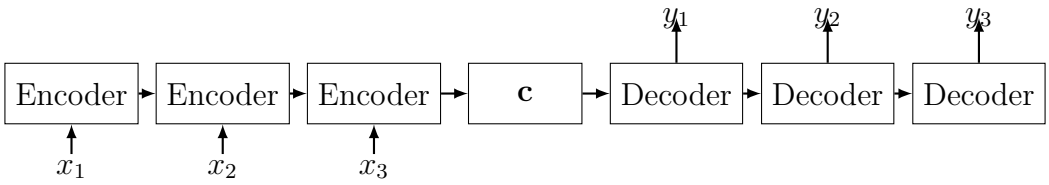
Los modelos clásicos (RNN, LSTM) pueden procesar secuencias pero no generan una nueva secuencia condicionada a la entrada. La arquitectura Seq2Seq, propuesta por Sutskever et al. (2014) [citation:9], aborda este problema con un enfoque de codificador-decodificador.

1.3. Arquitectura Encoder-Decoder

1.3.1. Visión general

La arquitectura Seq2Seq consta de dos redes recurrentes:

- **Encoder**: procesa la secuencia de entrada palabra por palabra y la comprime en un vector de contexto fijo **c** [citation:1][citation:5].
- **Decoder**: toma el vector de contexto como estado inicial y genera la secuencia de salida paso a paso, autoregresivamente (la salida anterior se usa como entrada para el siguiente paso) [citation:3][citation:6].



Arquitectura Seq2Seq básica. El encoder comprime la secuencia en **c**; el decoder genera la salida a partir de **c**.

1.3.2. Encoder

El encoder es una RNN (LSTM o GRU) que lee la secuencia de entrada $\mathbf{x} = (x_1, x_2, ..., x_T)$. En cada paso t, actualiza su estado oculto:

$$\mathbf{h}_t = \text{RNN}(\mathbf{x}_t, \mathbf{h}_{t-1})$$

El estado final $\mathbf{h}_T$ se utiliza como **vector de contexto c** [citation:1][citation:5]. En la práctica, suelen usarse LSTM y el vector de contexto puede ser el estado oculto final, una combinación de los estados o el estado de celda [citation:5].

1.3.3. Decoder

El decoder es otra RNN que genera la secuencia de salida $\mathbf{y} = (y_1, y_2, ..., y_{T'})$ paso a paso. Su estado inicial se establece con el vector de contexto: $\mathbf{s}_0 = \mathbf{c}$ [citation:3]. En cada paso t:

- Recibe la palabra generada en el paso anterior y_{t-1} (durante entrenamiento, se usa la palabra verdadera – **teacher forcing** [citation:1]).
- Actualiza su estado oculto: $\mathbf{s}_t = \text{RNN}(y_{t-1}, \mathbf{s}_{t-1})$.
- Produce una distribución de probabilidad sobre el vocabulario: $P(y_t|y_{<t}, \mathbf{c}) = \text{softmax}(\mathbf{W}_o\mathbf{s}_t + \mathbf{b}_o)$.

La generación continúa hasta que se produce el token especial de fin de secuencia (`<eos>`) [citation:3].

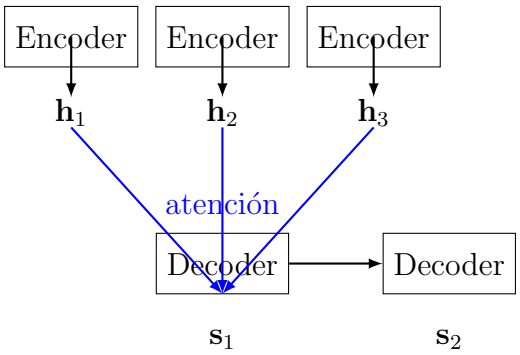
1.3.4. Limitación del contexto fijo

El vector de contexto $\mathbf{c}$ debe contener toda la información de la secuencia de entrada en una representación de dimensión fija. Para secuencias largas, esto se convierte en un cuello de botella: la información de las primeras palabras puede diluirse o perderse [citation:1][citation:4]. Además, diferentes partes de la salida pueden depender de diferentes partes de la entrada; un único vector no puede capturar todas estas alineaciones.

1.4. Mecanismo de Atención

1.4.1. Intuición

La atención permite que el decodificador, en cada paso de generación, "mire" directamente a todos los estados ocultos del encoder y decida qué partes de la entrada son más relevantes para predecir la siguiente palabra [citation:4][citation:8]. Esto elimina el cuello de botella del contexto fijo y mejora significativamente el rendimiento en secuencias largas.



El decodificador, en cada paso, calcula pesos de atención sobre todos los estados del encoder.

1.4.2. Atención de Bahdanau (Additive Attention)

Bahdanau et al. (2015) propusieron el primer mecanismo de atención para traducción automática [citation:4][citation:8]. En el paso t del decodificador, con estado oculto $\mathbf{s}_t$, se calcula un vector de pesos $\boldsymbol{\alpha}_t$ sobre los T estados del encoder $\mathbf{h}_1, ..., \mathbf{h}_T$.

1. **Cálculo de puntuaciones de alineación**:

$$e_{t,i} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{t-1} + \mathbf{U}_a \mathbf{h}_i)$$

   donde $\mathbf{v}_a$, $\mathbf{W}_a$ y $\mathbf{U}_a$ son matrices de pesos aprendibles. Esta puntuación mide qué tan bien el estado de entrada i y el estado de salida anterior se alinean.

2. **Normalización softmax**:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

   Los pesos $\alpha_{t,i}$ suman 1 e indican la relevancia de cada palabra de entrada para generar la palabra t.

3. **Vector de contexto ponderado**:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

   Este vector es la representación de la entrada enfocada en las partes relevantes.

4. **Combinación con el estado del decodificador**: El vector de contexto se concatena con el estado del decodificador $\mathbf{s}_t$ (o se usa para actualizar el estado) y luego se predice la palabra [citation:4].

1.4.3. Atención de Luong (Multiplicative Attention)

Luong et al. (2015) propusieron variantes más simples y eficientes. En lugar de usar $\mathbf{s}_{t-1}$ para calcular la atención, usan el estado actual $\mathbf{s}_t$ (después de actualizarlo con la entrada, pero antes de la capa de salida) [citation:8]. Las puntuaciones pueden calcularse de varias formas:

- **Producto punto**: $e_{t,i} = \mathbf{s}_t^\top \mathbf{h}_i$
- **Producto general**: $e_{t,i} = \mathbf{s}_t^\top \mathbf{W}_a \mathbf{h}_i$
- **Concat (similar a Bahdanau)**: $e_{t,i} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[\mathbf{s}_t; \mathbf{h}_i])$

Luego se procede con softmax y combinación como antes.

1.4.4. Atención global vs. local

- **Atención global**: considera todos los estados del encoder [citation:8]. Es más costosa computacionalmente pero captura toda la información.
- **Atención local**: predice una posición alineada en la entrada y solo considera una ventana alrededor de ella. Es más eficiente y puede ser igual de efectiva [citation:8].

1.5. Implementación paso a paso en PyTorch

1.5.1. Preparación de datos

Usaremos un pequeño conjunto de pares inglés-francés para traducción [citation:6][citation:10].

```
import torch
import torch.nn as nn
import torch.optim as optim
from collections import Counter
import numpy as np

# Datos de ejemplo
pairs = [
    ("hello", "bonjour"),
    ("world", "monde"),
    ("good morning", "bonjour"),
```

```
    ("how are you", "comment allez vous"),
    ("i love nlp", "j'aime le nlp"),
    ("good night", "bonne nuit"),
    ("see you later", "a plus tard"),
    ("thank you", "merci")
]

def build_vocab(sentences, max_size=50):
    word_counts = Counter()
    for sent in sentences:
        word_counts.update(sent.split())
    vocab = {word: idx+2 for idx, (word, _) in enumerate(
       word_counts.most_common(max_size-2))}
    vocab['<PAD>'] = 0
    vocab['<SOS>'] = 1
    vocab['<EOS>'] = len(vocab)
    return vocab

# Separar ingl s y franc s
eng_sents = [p[0] for p in pairs]
fra_sents = [p[1] for p in pairs]

eng_vocab = build_vocab(eng_sents)
fra_vocab = build_vocab(fra_sents)
eng_itos = {idx: word for word, idx in eng_vocab.items()}
fra_itos = {idx: word for word, idx in fra_vocab.items()}
```

1.5.2. Encoder

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
       num_layers=1, dropout=0.5):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
           batch_first=True, dropout=dropout)

    def forward(self, x):
        # x: (batch, seq_len)
        embedded = self.embedding(x)          # (batch, seq_len,
           embed_dim)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, hidden, cell
```

1.5.3. Decoder con Atención (Luong)

```
class DecoderWithAttention(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
       num_layers=1, dropout=0.5):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers,
            batch_first=True, dropout=dropout)
        self.attn_proj = nn.Linear(hidden_dim * 2, hidden_dim)
        self.fc_out = nn.Linear(hidden_dim, vocab_size)

    def forward(self, y_prev, hidden, cell, encoder_outputs):
        # y_prev: (batch, 1)
        # hidden, cell: (num_layers, batch, hidden_dim)
        # encoder_outputs: (batch, seq_len, hidden_dim)

        embedded = self.embedding(y_prev)  # (batch, 1, embed_dim
            )

        # LSTM step
        lstm_out, (hidden, cell) = self.lstm(embedded, (hidden,
            cell))  # lstm_out: (batch, 1, hidden_dim)

        # Calcular atenci n (producto punto)
        # lstm_out: (batch, 1, hidden_dim) -> (batch, hidden_dim)
        query = lstm_out.squeeze(1)  # (batch, hidden_dim)

        # encoder_outputs: (batch, seq_len, hidden_dim)
        # scores: (batch, seq_len)
        scores = torch.bmm(encoder_outputs, query.unsqueeze(2)).
            squeeze(2)
        attn_weights = torch.softmax(scores, dim=1)  # (batch,
            seq_len)

        # Vector de contexto ponderado: (batch, hidden_dim)
        context = torch.bmm(attn_weights.unsqueeze(1),
            encoder_outputs).squeeze(1)

        # Combinar con el estado LSTM
        combined = torch.tanh(self.attn_proj(torch.cat((query,
            context), dim=1)))  # (batch, hidden_dim)

        # Predicci n
        output = self.fc_out(combined)  # (batch, vocab_size)
        return output, hidden, cell, attn_weights
```

1.5.4. Modelo Seq2Seq completo

```
class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder, device):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.device = device
```

```
    def forward(self, src, trg, teacher_forcing_ratio=0.5):
        batch_size, trg_len = trg.shape
        trg_vocab_size = self.decoder.fc_out.out_features

        # Tensor para guardar las salidas
        outputs = torch.zeros(batch_size, trg_len, trg_vocab_size
           ).to(self.device)

        # Encoder
        encoder_outputs, hidden, cell = self.encoder(src)

        # Primer input al decoder: token <SOS>
        decoder_input = torch.ones(batch_size, 1).long().to(self.
           device)  # <SOS> = 1

        for t in range(trg_len):
            output, hidden, cell, attn = self.decoder(
               decoder_input, hidden, cell, encoder_outputs)
            outputs[:, t, :] = output

            # Teacher forcing
            if torch.rand(1).item() < teacher_forcing_ratio:
                decoder_input = trg[:, t].unsqueeze(1)
            else:
                decoder_input = output.argmax(1).unsqueeze(1)

        return outputs
```

1.5.5. Entrenamiento

```
device = torch.device('cuda' if torch.cuda.is_available() else '
   cpu')

# Hiperpar metros
EMBED_DIM = 64
HIDDEN_DIM = 128
NUM_LAYERS = 1
DROPOUT = 0.5

encoder = Encoder(len(eng_vocab), EMBED_DIM, HIDDEN_DIM,
   NUM_LAYERS, DROPOUT).to(device)
decoder = DecoderWithAttention(len(fra_vocab), EMBED_DIM,
   HIDDEN_DIM, NUM_LAYERS, DROPOUT).to(device)
model = Seq2Seq(encoder, decoder, device).to(device)

criterion = nn.CrossEntropyLoss(ignore_index=0)  # ignorar
   padding
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
# Datos de ejemplo (convertir a  ndices )
def encode_sentence(sent, vocab, max_len=10):
    tokens = sent.split()
    indices = [vocab.get(t, 1) for t in tokens]  # <UNK> no
       definido, usamos <SOS> como fallback
    indices = [vocab['<SOS>']] + indices + [vocab['<EOS>']]
    # padding
    indices += [vocab['<PAD>']] * (max_len + 2 - len(indices))
    return indices[:max_len+2]

X = torch.tensor([encode_sentence(p[0], eng_vocab) for p in pairs
   ])
Y = torch.tensor([encode_sentence(p[1], fra_vocab) for p in pairs
   ])

# Entrenamiento (simplificado)
epochs = 100
for epoch in range(epochs):
    optimizer.zero_grad()
    output = model(X, Y)
    loss = criterion(output.reshape(-1, len(fra_vocab)), Y.
       reshape(-1))
    loss.backward()
    optimizer.step()
    if epoch % 20 == 0:
        print(f'Epoch {epoch}, Loss: {loss.item():.4f}')
```

1.6. Métrica de Evaluación: BLEU

1.6.1. Definición

BLEU (Bilingual Evaluation Understudy) es la métrica estándar para evaluar traducción automática [citation:8]. Mide la similitud entre una traducción generada y una o más referencias humanas. Se basa en la precisión de n-gramas, con una penalización por longitud (brevity penalty).

1.6.2. Cálculo

Para una traducción candidata y una referencia:

1. Se calcula la precisión de n-gramas para $n = 1, 2, 3, 4$:

$$p_n = \frac{\#\text{ n-gramas candidatos que aparecen en la referencia}}{\#\text{ n-gramas en el candidato}}$$

2. Se combinan las precisiones:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right)$$

donde típicamente $w_n = 1/4$.

3. La penalización por longitud (brevity penalty) evita que traducciones cortas obtengan puntuaciones altas:

$$\text{BP} = \begin{cases} 1 & \text{si } c > r \\ e^{(1-r/c)} & \text{si } c \leq r \end{cases}$$

con c longitud del candidato, r longitud de la referencia.

1.6.3. Ejemplo numérico

Referencia: "the cat sits on the mat" Candidato: "the cat on the mat"

- Unigramas: candidato the, cat, on, the, mat; aciertos: the (2), cat (1), on (1), mat (1) → 5/5 = 1.0
- Bigramas: the cat, cat on, on the, the mat; aciertos: the cat (1), on the (1) → 2/4 = 0.5
- BLEU (sin BP): exp(0.25*log(1) + 0.25*log(0.5)) = exp(0.25*(-0.693)) = exp(-0.173) = 0.841
- Como c=5, r=6, BP = e$^{(1-6/5)}$ = $e^{-0.2}$ = 0.819
- BLEU final = 0.819 * 0.841 = 0.689

1.6.4. Interpretación

BLEU varía entre 0 y 1, siendo 1 una coincidencia perfecta. En la práctica, puntuaciones ¿0.3 indican traducciones comprensibles, ¿0.5 son muy buenas [citation:8].

1.7. Aplicaciones en la industria

1.7.1. Traducción automática

Google Translate y DeepL utilizaron arquitecturas Seq2Seq con atención antes de los Transformers. Hoy en día, los modelos basados en atención (Transformers) son el estándar, pero la idea fundamental proviene de estos mecanismos.

1.7.2. Summarization abstractivo

Modelos como BART y T5, basados en Transformers, realizan summarization generando resúmenes a partir de documentos largos. El mecanismo de atención permite que el decodificador se enfoque en las partes relevantes del texto original.

1.7.3. Diálogo y chatbots

Los sistemas de diálogo (como los de atención al cliente) utilizan secuencias de preguntas y respuestas. La atención ayuda a mantener el contexto de la conversación [citation:8].

1.7.4. Generación de preguntas (QA)

Dado un párrafo y una respuesta, el modelo genera una pregunta adecuada. La atención alinea la respuesta con el contexto.

1.8. Preparación para entrevistas

1.8.1. Preguntas típicas

- **"¿Cómo funciona el mecanismo de atención?"** Se calculan pesos sobre los estados del encoder usando una función de alineación (p.ej., producto punto o red pequeña), se normalizan con softmax, y se obtiene un vector de contexto ponderado que se combina con el estado del decodificador para predecir la siguiente palabra.

- **"¿Cuál es la diferencia entre atención de Bahdanau y Luong?"** Bahdanau usa el estado anterior del decodificador $\mathbf{s}_{t-1}$ para calcular la atención, mientras que Luong usa el estado actual $\mathbf{s}_t$ después de actualizar con la entrada. Luong también propone variantes más simples (producto punto, general).

- **"¿Qué es teacher forcing y por qué se usa?"** Técnica de entrenamiento donde se alimenta al decodificador con la palabra verdadera en lugar de su propia predicción. Acelera la convergencia pero puede causar exposure bias (diferencia entre entrenamiento e inferencia) [citation:1].

- **"Explica la métrica BLEU."** Mide la precisión de n-gramas entre la traducción candidata y referencias humanas, con una penalización por longitud. Es la métrica estándar en traducción automática.

- **"¿Por qué el contexto fijo de Seq2Seq básico es problemático?"** Para secuencias largas, el vector de contexto fijo se convierte en un cuello de botella; la información de las primeras palabras puede perderse. La atención resuelve esto permitiendo acceso directo a todos los estados del encoder.

1.8.2. Preguntas avanzadas

- **"¿Qué es beam search y cómo mejora la generación?"** En lugar de tomar greedy la palabra más probable en cada paso, beam search mantiene k hipótesis (caminos) y las expande, seleccionando al final la de mayor probabilidad conjunta. Reduce el riesgo de elegir un mal camino temprano [citation:1][citation:8].

- **"¿Cómo manejarías secuencias de longitud variable en un lote?"** Usar padding con token especial y luego usar 'pack$_{p}added_{s}equence$'$enPyTorchparaquelaRNNignorelospas$ 6].

- **"¿Qué es el problema de exposure bias?"** Durante entrenamiento, con teacher forcing, el modelo ve siempre la palabra correcta; en inferencia, ve sus propias predicciones (que pueden ser erróneas). Esto puede causar que los errores se acumulen. Soluciones: scheduled sampling o entrenamiento con secuencias generadas.

1.9. Conexión con el resto del curso

- **Sesión 7 (RNN/LSTM)**: La base de los encoders/decoders.
- **Sesión 9 (Transformers)**: La atención es el pilar de los Transformers, que eliminan la recurrencia y usan solo atención.
- **Sesiones anteriores**: Las capas de embedding y regularización son comunes.

1.10. Resumen

- Los modelos **Seq2Seq** con encoder-decoder transforman secuencias de longitud variable.
- El **vector de contexto fijo** limita el rendimiento en secuencias largas.
- El **mecanismo de atención** permite al decodificador enfocarse en partes relevantes de la entrada, mejorando la traducción y el manejo de dependencias largas.
- Hay dos formulaciones principales: atención **aditiva (Bahdanau)** y **multiplicativa (Luong)**.
- La métrica **BLEU** evalúa la calidad de la traducción basándose en n-gramas y penalización por longitud.
- Estos conceptos son la base de los modelos modernos como Transformers.